# Canvas > Discussions >

*Looking for a Partner for the Home Assignments?*

# Volunteer To Be Student Representatives Please!

email: elenap@chalmers.se [TODAY!]

# TDA352/DIT352 CRYPTOGRAPHY

## Module 1 : Lecture 2 - Agenda

**Pseudorandom Generators (PRG)**

- Security (Real or Random?)
- Secure encryption from PRG
- Semantic Security (Left or Right)
- PRG-Based Encryption [Proof]

**Block Ciphers**

- Pseudo Random Permutations
- Definition of Block Cipher
- Chosen Plaintext Attack (IND-CPA)
- AES (Advanced Encryption Standard)

**Modes of Operation**

- ECB, CBC, CTR
- Is AES-ECB IND-CPA?
- Message Padding

# Recap From the Last Lecture

**Kerckhoffs's Principle**

Some **definitions**, **notations**, and important **terminology**

**Perfect Secrecy**

*several equivalent definitions*

**Symmetric Encryption** (and **CORRECTNESS**)

**PROOF**

The OTP is perfectly secure

**The One Time Pad (OTP).** $c = k \oplus m$

*Both proofs hinge on the fact that there exists a unique (or same nr of) key that maps any one plaintext to any one ciphertext*

**PRG**

**PROOF**

**Shannon's Theorem**

perfect security $\Rightarrow |\mathcal{K}| \geq |\mathcal{M}|$

**How close to perfect security can we go, while being practical?**

# TDA352/DIT352 CRYPTOGRAPHY

## Module 1 : Lecture 2 - Agenda

**Pseudorandom Generators (PRG)**

- Security (Real or Random?)
- Secure encryption from PRG
- Semantic Security (Left or Right)
- PRG-Based Encryption [Proof]

**Block Ciphers**

- Pseudo Random Permutations
- Definition of Block Cipher
- Chosen Plaintext Attack (IND-CPA)
- AES (Advanced Encryption Standard)

**Modes of Operation**

- ECB, CBC, CTR
- Is AES-ECB IND-CPA?
- Message Padding

# Pseudo Random Generators (PRG)

A pseudo random generator is a **deterministic**, efficient function PRG that **stretches** a short random input $s$ into a longer output.

*(small amount of **true** randomness → large amount of **pseudo** randomness)*

**No efficient algorithm** should be able to tell $\text{PRG}(s)$ apart from a truly uniform string of the same length, except with **negligible** advantage.

A Pseudo Random Generator is a deterministic function $\text{PRG} : \{0,1\}^n \to \{0,1\}^\ell$ such that:

- $\text{PRG}(\cdot)$ is efficiently computable
- $\text{PRG}(\cdot)$ expands its input, i.e., $\ell > n$
- For a random input seed $s \leftarrow \$\{0,1\}^n$ no efficient adversary can tell apart the output of $\text{PRG}(s)$ from a random string $k \leftarrow \$\{0,1\}^\ell$.



# “Real OR Random” Security (Intuition)





https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.youtube.com%2Fwatch%3Fv%3DzGLx_mLLu0c&psig=AOvVaw2apYsTgLReXypLHAZ3jP-3&ust=1650024734701000&source=images&cd=vfe&ved=2ahUKEwixrN2iw5P3AhXYZfEDHcXCDJcQjhx6BAgAEAw

# Security Game For PRG

**Aim**: quantify the attacker's likelihood in distinguishing PRG from a source of uniform randomness over $\{0,1\}^{\ell}$

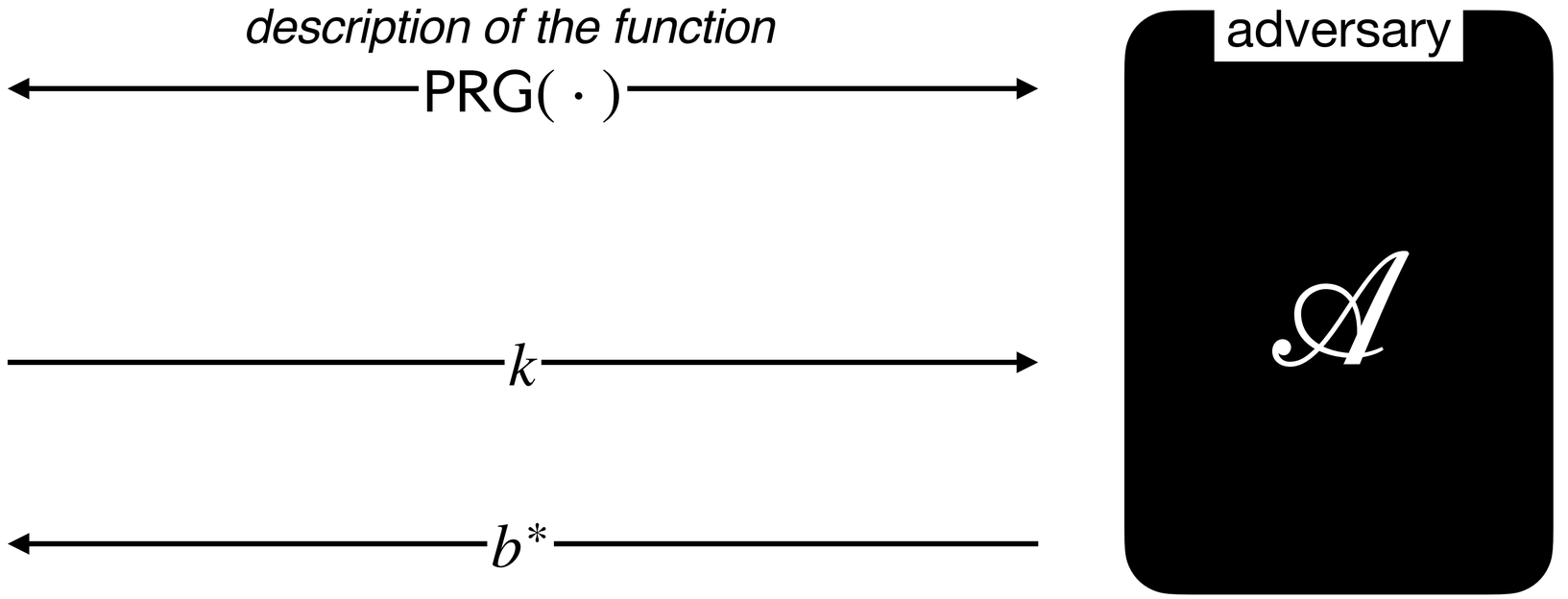
challenger $\mathscr{C}$

$b \leftarrow \$\{0,1\}$

If $b = 0$:
$s \leftarrow \$\{0,1\}^n$
$k = \text{PRG}(s)$

If $b = 1$:
$k \leftarrow \$\{0,1\}^{\ell}$

If $b^* = b$ : $\mathscr{A}$ **wins**
else : $\mathscr{A}$ **loses**

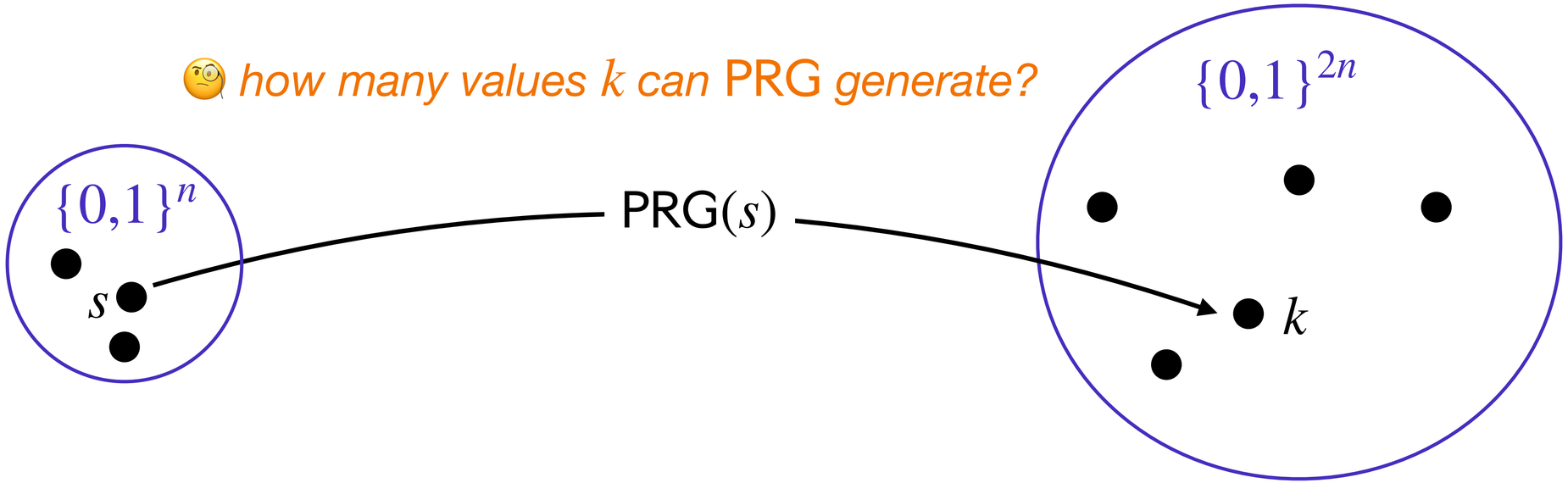



A function PRG : $\{0,1\}^n \rightarrow \{0,1\}^{\ell}$ is a secure pseudo random function if any probabilistic polynomial time attacker $\mathscr{A}$ has only negligible advantage in winning the PRG security game. Formally,

$$Adv(\mathscr{A}) = |Pr[\mathscr{A}\ wins] - \frac{1}{2}| < negl(n)$$

# Note: PRG Outputs Are Far From Random

Consider a secure PRG : $\{0,1\}^n \rightarrow \{0,1\}^{2n}$



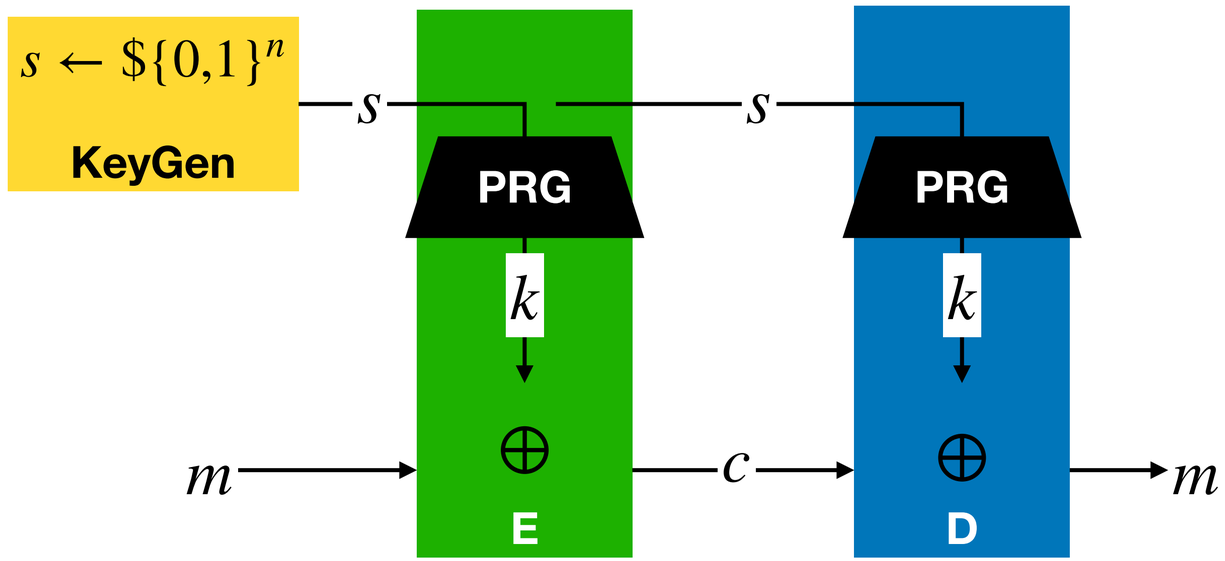

The size of the image of a function is at most as large as its input space.
So PRG cannot generate more than $2^n$ many $2n$-bit strings (half the space!).

🎯 Setting $n = 128$ or larger prevents brute force distinguishers

*assuming $\mathcal{A}$ cannot do more than $2^{80}$ CPU cycles (evaluations) during the lifetime of the PRG*

# Constructing a Secure Encryption Scheme From a PRG

Let PRG : $\{0,1\}^n \rightarrow \{0,1\}^\ell$ be a secure PRG.




**A One-time PRG cipher**

$\mathcal{M} = C = \{0,1\}^\ell,\ \mathcal{K} = \{0,1\}^n,\ n < \ell$

$KeyGen(1^n) \rightarrow s\ (where\ s \leftarrow \$\{0,1\}^n)$

$Enc(s, m) = \text{PRG}(s) \oplus m$

$Dec(s, c) = \text{PRG}(s) \oplus c$

🧐 *Is this cipher perfectly secure?*

No! We need a **new security definition** that works when $|\mathcal{K}| < |\mathcal{M}|$

# “Left OR Right” Security (Intuition)



# Semantic Security (Single-Message Secure Encryption)

**Aim**: quantify the attacker's likelihood in distinguishing an encryption of a (chosen) message $m_0$ from an encryption of another (chosen) message $m_1$

challenger

$\mathscr{C}$

$k \leftarrow KeyGen(1^n)$

$b \leftarrow \$\{0,1\}$

If $b = 0$:
$c \leftarrow E(k, m_0)$
If $b = 1$:
$c \leftarrow E(k, m_1)$

If $b^* = b$ : $\mathscr{A}$ **wins**
else : $\mathscr{A}$ **loses**

$\longleftarrow (KeyGen,\ E,\ D) \longrightarrow$

$\longleftarrow (m_0, m_1) \quad s.t.\ |m_0| = |m_1| \longrightarrow$

$\longrightarrow c \longrightarrow$

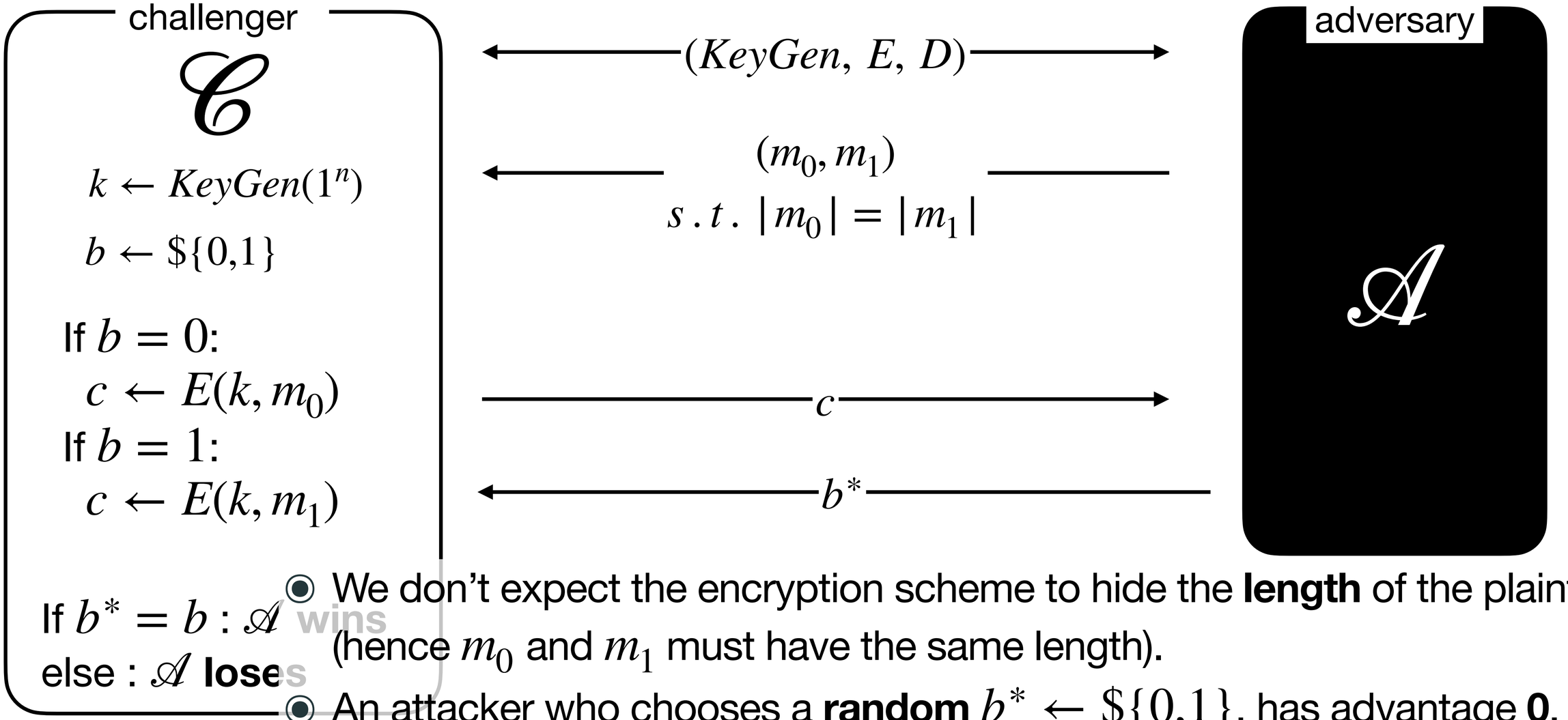
$\longleftarrow b^* \longleftarrow$

adversary

$\mathscr{A}$

A symmetric encryption scheme is **semantically secure** if any efficient attacker $\mathscr{A}$ has only negligible advantage in winning the semantic security game: $Adv(\mathscr{A}) = |Pr[\mathscr{A}\ wins] - \frac{1}{2}| < negl(n)$

# Remarks on the Definition

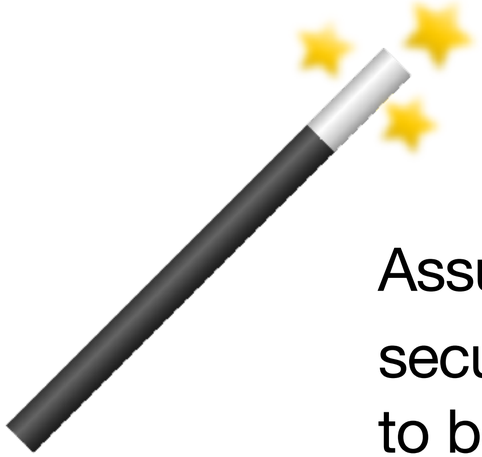



- We don't expect the encryption scheme to hide the **length** of the plaintext; (hence $m_0$ and $m_1$ must have the same length).
- An attacker who chooses a **random** $b^* \leftarrow \$\{0,1\}$, has advantage **0**.
- An attacker who **always answers** $b^* = 1$ (or $b^* = 0$) also has advantage **0**.
- If the encryption scheme is the **one time pad**, any attacker has advantage **0**.

# Proving our Construction Is Semantically Secure

**Theorem** (secure encryption scheme with short keys)
If PRG : $\{0,1\}^n \rightarrow \{0,1\}^\ell$ is a secure PRG, then the cipher defined by
$Enc(s,m) = \text{PRG}(s) \oplus m$; $Dec(s,c) = \text{PRG}(s) \oplus c$ is semantically secure.

**Proof Plan:**
We must prove that any efficient adversary against the encryption's semantical security has negligible advantage, without knowing anything about the adversary's strategy.



🧐 *HOW ?*

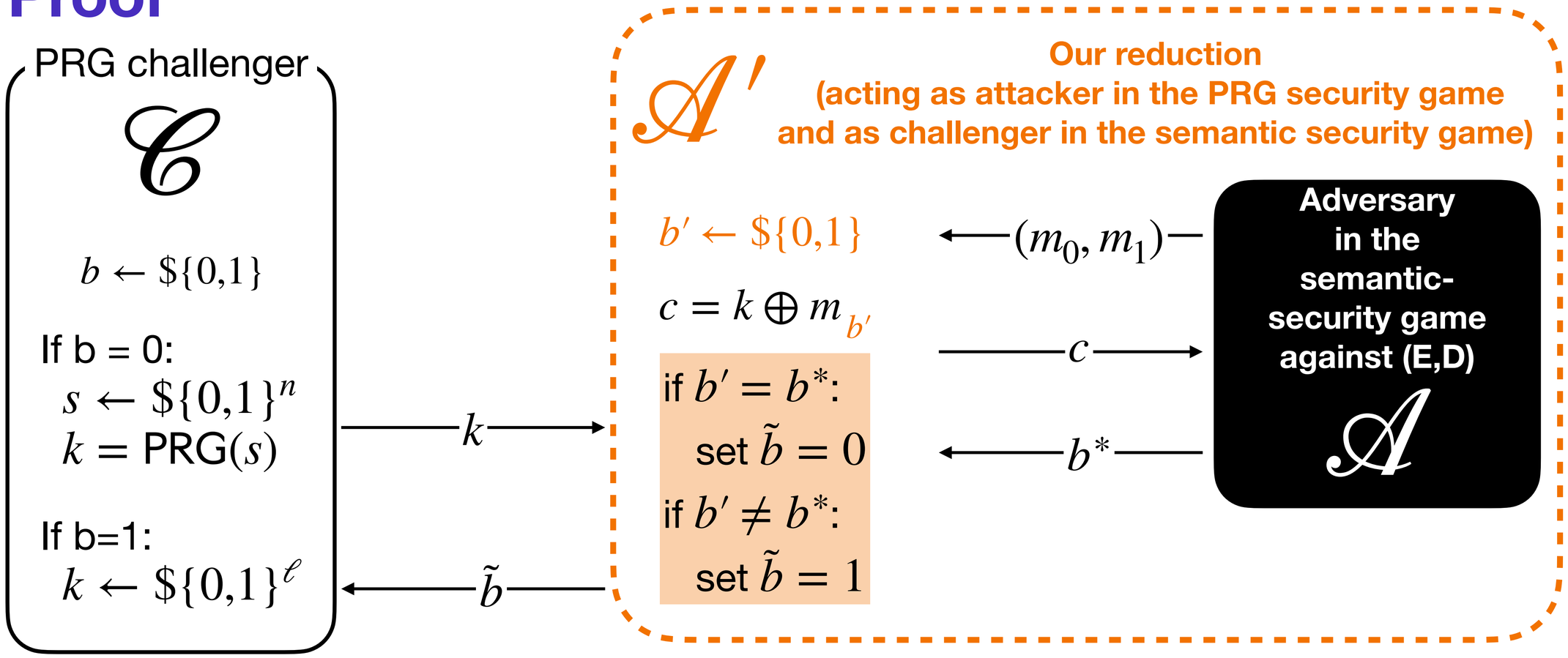
…or… ***proof by reduction to absurd***

Assume that there exists an adversary $\mathscr{A}$ that can break the semantic security of the encryption. Then we build a new adversary $\mathscr{A}'$ that uses $\mathscr{A}$ to break the security of the PRG. Since PRG is assumed to be secure, such $\mathscr{A}'$ cannot exist. Thus it was absurd to assume $\mathscr{A}$ exists in the first place.

# The Proof

This proof [next 3 slides] is not part of the exam, but it is here to show you how “formal reduction proofs” look like. If you understand the mechanism and like the reasoning, then you have a good chance to become a cryptographer!

# The Proof




## Important observations

If $b = 0$, the ciphertext $c$ is the encryption using the PRG cipher. Because we assumed that $\mathscr{A}$ wins this game with non negligible probability this means $b' = b^*$. So $\mathscr{A}'$ wins when $\mathscr{A}$ does.

If $b = 1$, $\mathscr{A}'$ encryption is the **OTP** (perfectly secure), thus $\mathscr{A}$ has no advantage. So $\mathscr{A}'$ only guesses correctly with 1/2 probability (0 advantage).

# The Proof

*complementary events*

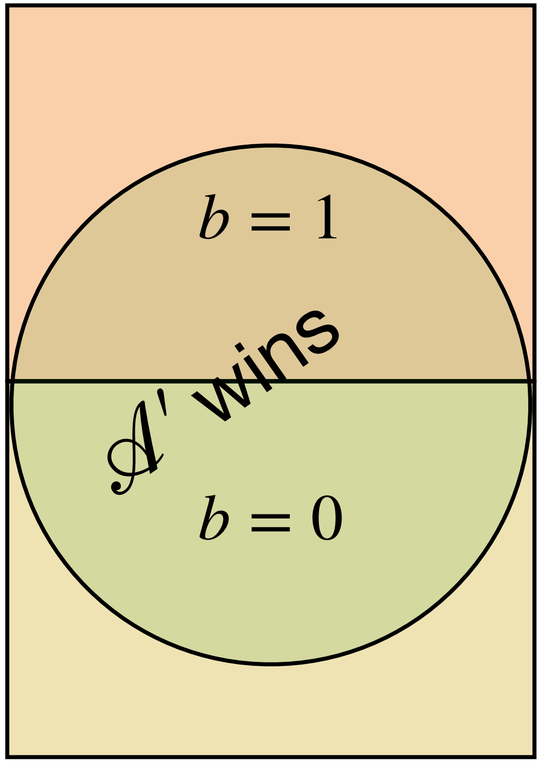
Pr[$\mathcal{A}'$ wins] = Pr[$\mathcal{A}'$ wins AND $b = 0$] + Pr[$\mathcal{A}'$ wins AND $b = 1$]

**conditional probability** $Pr[A \mid B] = \frac{Pr[A \cap B]}{Pr[B]}$

= Pr[$\mathcal{A}'$ wins | $b = 0$] Pr[b=0] + Pr[$\mathcal{A}'$ wins | $b = 1$] Pr[b=1]

*PRG-cipher* **1/2** *OTP-cipher (perfect security)* **1/2**

**= Pr[$\mathcal{A}$ wins sem.sec game against (E,D)]**




**Thus Pr[$\mathcal{A}'$ wins PRG] = Pr[$\mathcal{A}$ wins sem.sec] · (1/2) + 1/2 · (1/2)**

Or, reorganising the terms: Pr[$\mathcal{A}$ wins sem.sec] = 2 Pr[$\mathcal{A}'$ wins PRG] - 1/2

$$Adv_{sem.sec}(\mathcal{A}) = |Pr[\mathcal{A}\ wins] - \frac{1}{2}| = |(2Pr[\mathcal{A}'\ wins\ PRG] - 1/2) - \frac{1}{2}| = 2 \cdot Adv_{PRG}(\mathcal{A}')$$

# The Proof

$$Adv_{sem.sec}(\mathscr{A}) = |Pr[\mathscr{A}\ wins] - \frac{1}{2}| = |(2Pr[\mathscr{A}'\ wins\ PRG] - 1/2) - \frac{1}{2}| = 2 \cdot Adv_{PRG}(\mathscr{A}')$$

This concludes the proof of the theorem ⬆️

🧐 *WHY ?*

If our PRG-based encryption is not secure, then $\mathscr{A}$ has a non-negligible advantage in winning the semantic security game. If that was the case, we have constructed an efficient (PPT) reduction/adversary $\mathscr{A}'$ that uses $\mathscr{A}$ to win the PRG security game and succeed with half of the advantage of $\mathscr{A}$. Since we assumed the PRG to be secure, it is impossible for any efficient adversary to break the PRG. Therefore, such an $\mathscr{A}'$ cannot exist. Which in turn implies that $\mathscr{A}$ cannot exist. So it was absurd to assume such an $\mathscr{A}$ existed in the first place.

This reasoning implies that our PRG-based encryption is *provably* secure.

# Recap Thus Far

- unconditionally secure encryption exists, but it's impractical (perfect secrecy with the one time pad cipher, Shannon's theorem)
- In cryptography we want something practical so we need to look for meaningful conditions under which we can prove security
-  : we can expand a short randomness to a long one! PRG!
- PRG-based OTP cipher is semantically secure, i.e., to any PPT attacker $\mathscr{A}$ an encryption of "0" is indistinguishable from an encryption of "1".

# We’ve got secure communication!

🧐 ***Are we done yet?***

- Multi-Message Security
- Messages of different length
- Message integrity (Lecture 4)
- Shared secret key (Module 2)

# TDA352/DIT352 CRYPTOGRAPHY

## Module 1 : Lecture 2 - Agenda

**Pseudorandom Generators (PRG)**

- Security (Real or Random?)
- Secure encryption from PRG
- Semantic Security (Left or Right)
- PRG-Based Encryption [Proof]

**Block Ciphers**

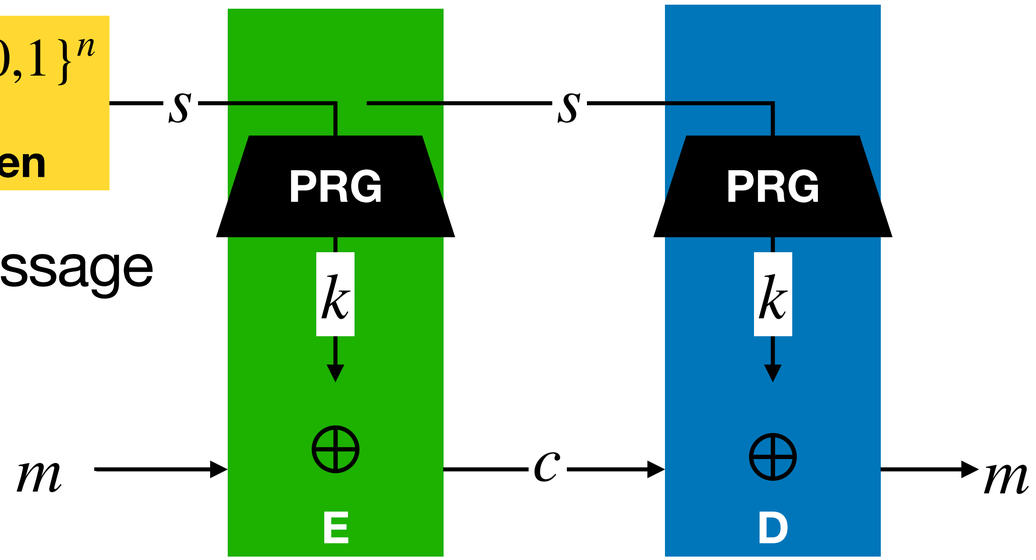
- Pseudo Random Permutations
- Definition of Block Cipher
- Chosen Plaintext Attack (IND-CPA)
- AES (Advanced Encryption Standard)

**Modes of Operation**

- ECB, CBC, CTR
- Is AES-ECB IND-CPA?
- Message Padding

# A Candidate for Multi-Message Secure Encryption

One-Message Secure encryption (OTP+PRG)




1. Define encryption as $E(s, m) =$

$= (s, \text{PRG}(s) \oplus m) = c$

for a random $s \leftarrow \$\{0,1\}^n$, different for every message

🤔 *There's a problem…*

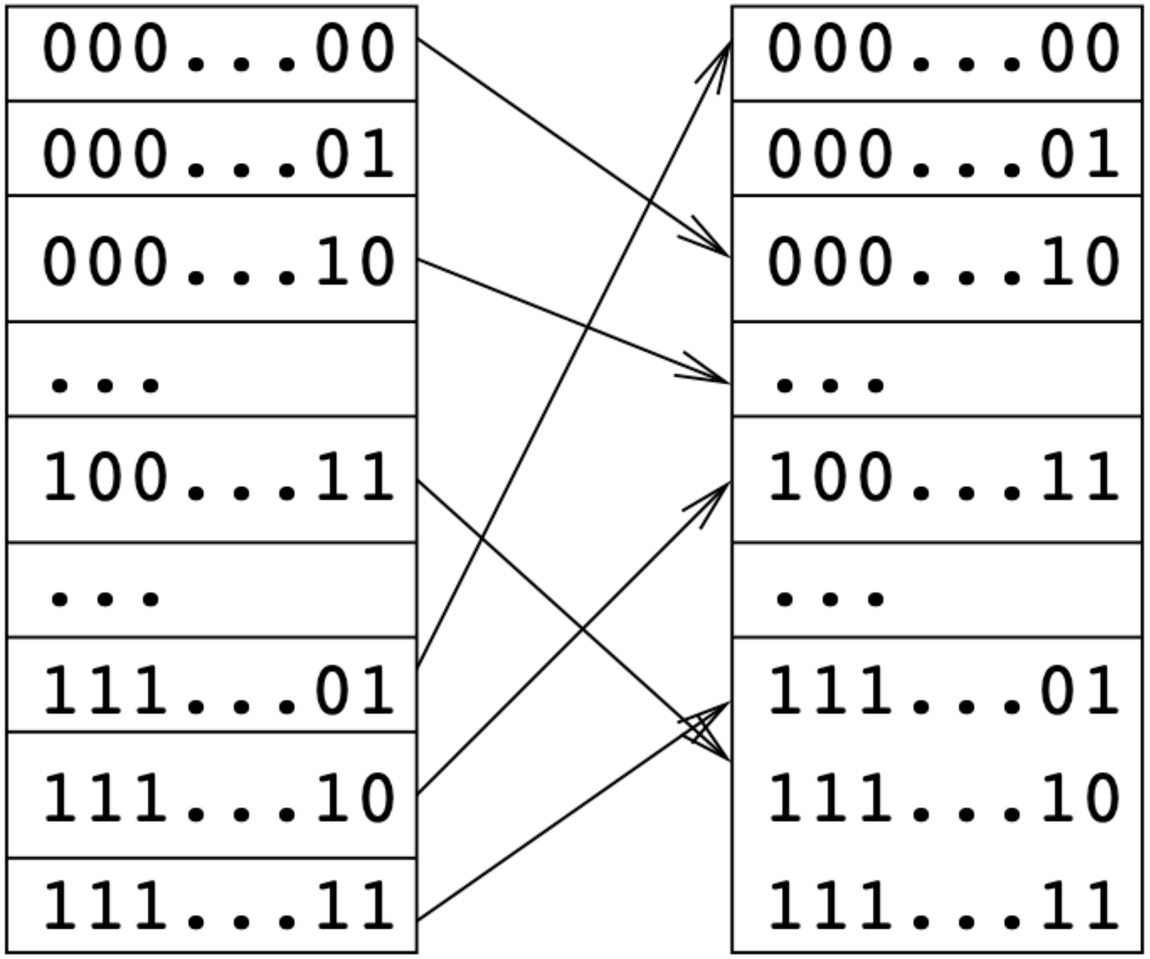
2. Find a way to make the PRG depend on a shared key.

This object is called Pseudo Random Permutation $f_k : \{0,1\}^n \rightarrow \{0,1\}^n$.

Encryption is defined as $c = (s, f_k(s) \oplus m)$ for a random $s$ (different for every message).

```
KeyGen(1^K) : k ←${0,1}^K
E(k,m)      : Pick s ←${0,1}^n. Output c=(s, f_k(s) ⊕ m)
D(k,c)      : Output c[1] ⊕ f_k(c[0])
```

# What Is a Pseudo Random Permutation (PRP)?




A **random permutation** is a permutation chosen uniformly at random from the set Perms($\mathscr{M}$) of all permutations (one-to-one maps) on the set $\mathscr{M} = \{0,1\}^n$.

Informal:

A family of **pseudo-random permutations** $\{f_k : \{0,1\}^n \to \{0,1\}^n\}_{k \in \{0,1\}^K}$ is a collection of permutations such that:

1) each $f_k$ can be *efficiently computed;* and
2) there is *no efficient way to distinguish* between $f_k$ and a random function.

🧐 *How many possible permutations over $\mathscr{M} = \{0,1\}^n$ are there? |Perms($\mathscr{M}$)|?*

|Perms($\mathscr{M}$)| = $2^n! \approx 2^{n2^n}$ vs |PRP| = $|\{0,1\}^K| = 2^K$

# Pseudo Random Permutation

```
KeyGen(1^K) : k ←${0,1}^K
E(k,m)      : Pick s ←${0,1}^n. Output c=(s, f_k(s) ⊕ m)
D(k,c)      : Output c[1] ⊕ f_k(c[0])
```

**Theorem**

If $f_k$ is a PRP then the scheme (KeyGen, E, D) is a secure encryption scheme.

we need a *new* definition of security for multi messages

Let's *refine* the definition of an encryption scheme

# Block Ciphers

A Block Cipher is a **keyed** function that is **deterministic** and **invertible**.

Formally, $E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$, where $\mathcal{K} = \{0,1\}^K$ is a key space, $\mathcal{M} = \{0,1\}^n$ is the block space and for every $k \in \mathcal{K}$, the function $E(k, \cdot) : \{0,1\}^n \rightarrow \{0,1\}^n$ is deterministic, and invertible hence there exists an efficient (deterministic) function $D(k, \cdot) : \{0,1\}^n \rightarrow \{0,1\}^n$ such that $D(k, E(k, m)) = m$ for every $m \in \{0,1\}^n$.

Plaintexts and ciphertexts are both called **blocks**.

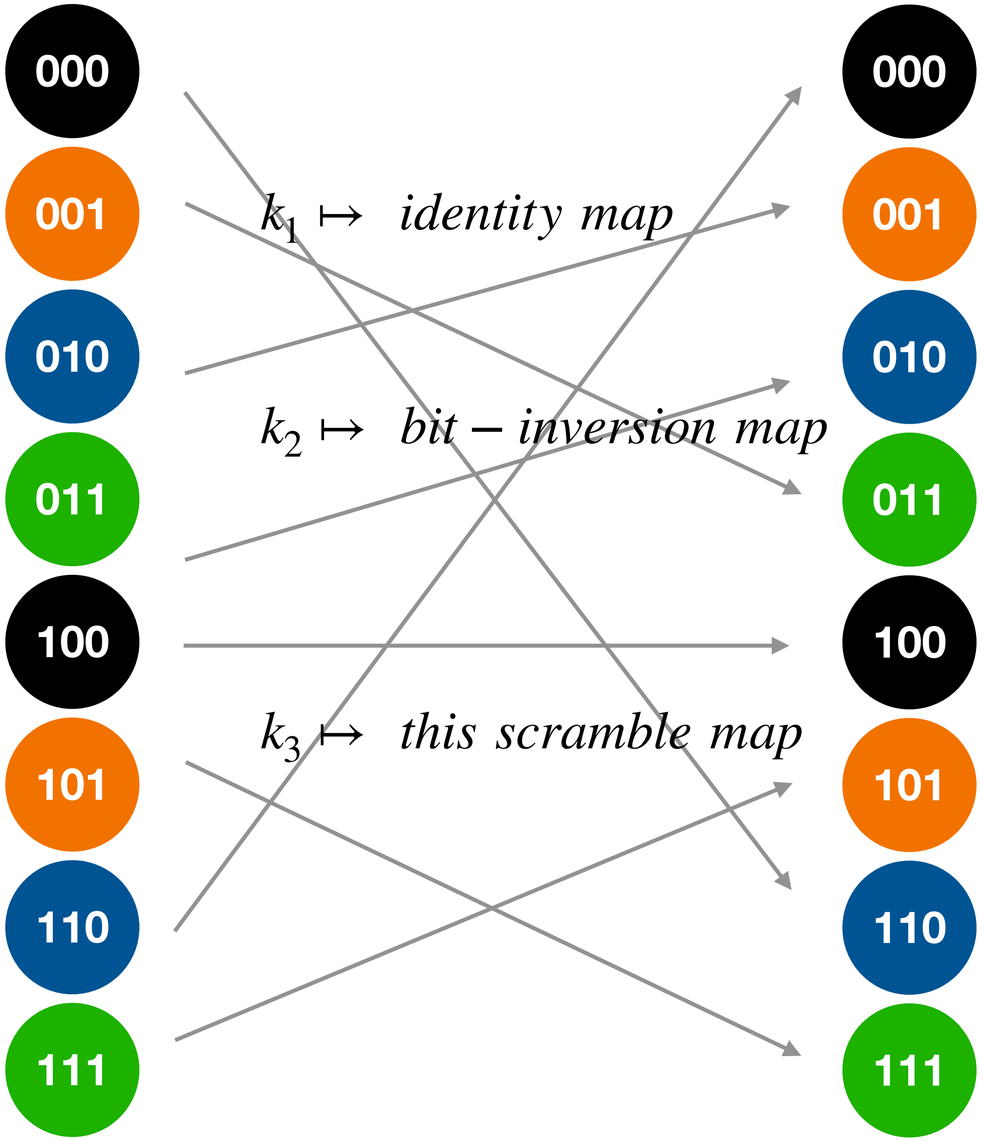
🧐 *What if the plaintext message is longer than one block (n-bits) ?*

The Block cipher “chains” blocks according to a “mode of operation” (more on this later)

**Observation**: we can now reuse the same key to encrypt/decrypt multiple messages!

# Block Ciphers - Examples

$\{0,1\}^{n=3} \; - - \; E(k,\cdot) - \rightarrow \; \{0,1\}^{n=3}$




🧐 *How many possible keys (encryption maps) are there?*

The total number of invertible functions over $\{0,1\}^3$ is

$8! = 8 \cdot 7 \cdots \cdot 2 \cdot 1 = 40\,320$

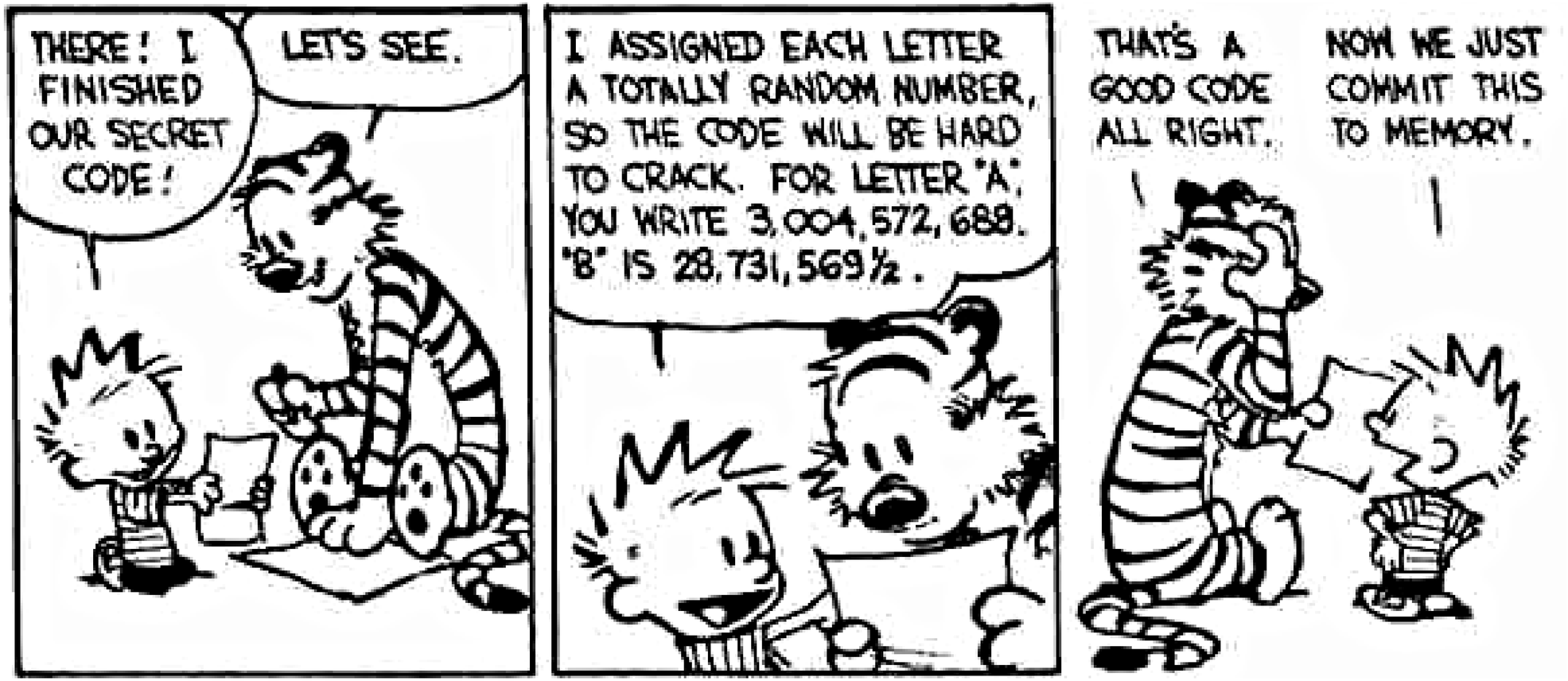
Large enough?!



**Bad news**: $8! \sim 2^{15}$ is manageable for modern computers

Typical values today are $|\mathcal{K}| = 2^{128}$ *or* $2^{256}$

# Security vs Efficiency - Finding the Right Balance




We cannot (efficiently) implement (all possible) random permutations for reasonable sizes of $n$.

Instead, we strive to construct ciphers that **cannot be distinguished from random permutations**.

# Towards a New Security Notion




**Adversary's Goal**

~~To decrypt a ciphertext~~

*Security can be damaged with much less*

~~To gain some information about the plaintext concealed in the ciphertext~~

*Vague, we would need to quantify this leakage (possible but..)*

To *distinguish* between the encryption of two *known* plaintext messages

In crypto jargon: **indistinguishability under chosen plaintext attack (IND-CPA)**

*IND-CPA has many equivalent notions, read here if you're interested*

**Historical example**: British military would place mines in particular locations hoping Germans would send encrypted messages about that location.

**Modern example**: Attacker-controlled Javascript on a web page causes victim web client to make a HTTPS connection.

# Security Notions for Block Ciphers




### Adversary's Goal

To *distinguish* between the encryption of two *known* plaintext messages
In crypto jargon: **indistinguishability under chosen plaintext attack (IND-CPA)**

### Adversary's Power

Efficient algorithm (probabilisitic, and runs in polynomial time $<2^{80}$)

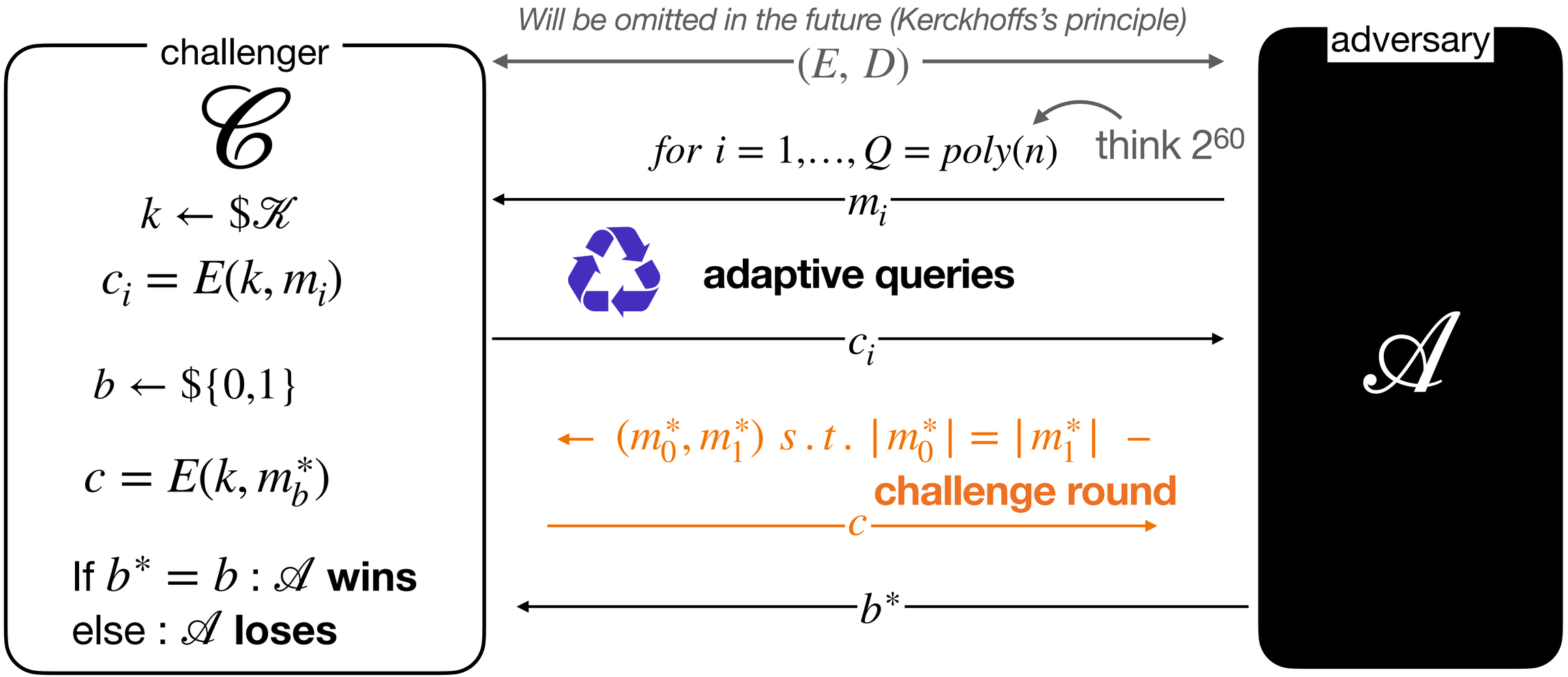
$\mathscr{A}$ can see everything transmitted over the communication channel

$\mathscr{A}$ knows all details of the encryption scheme except for the secret key (**Kerckhoffs's *principle***)

*"Security through obscurity" is an obsolete mantra*
*[computers are good for reverse-engineering, hackers are clever]*

# Indistinguishability Under Chosen Plaintext Attack (IND-CPA)

**Aim**: quantify the $\mathscr{A}$'s likelihood in distinguishing between encryptions of two messages, given that $\mathscr{A}$ can see encryptions of any messages of its choice.




An encryption scheme is **IND-CPA secure** if any efficient attacker $\mathscr{A}$ has only negligible advantage in winning the IND-CPA game.

# The Block Cipher *Par Excellence:* AES

## Advanced Encryption Standard

```
function AES_K(M)
(K_0,...,K_10) ← expand(K)
s ← M⊕K_0

for r = 1 to 10 do
  s ← substitute-nibbles(s)
  s ← shift-rows(s)
  if (r≤9) then
    s ← mix-cols(s)
  fi
  s ← s ⊕ K_r
endfor
return s
```

*key of **reduced** size*

*reuse material*

*OTP-style **recycle***

Other modern secure block ciphers: **PRESENT** (IoT devices), **Camellia** (Japan's standard, 128-bit blocks, 128/192/256-bit keys), **SM4** (China's standard 128-bit blocks and keys), **Triple-DES** (legacy).

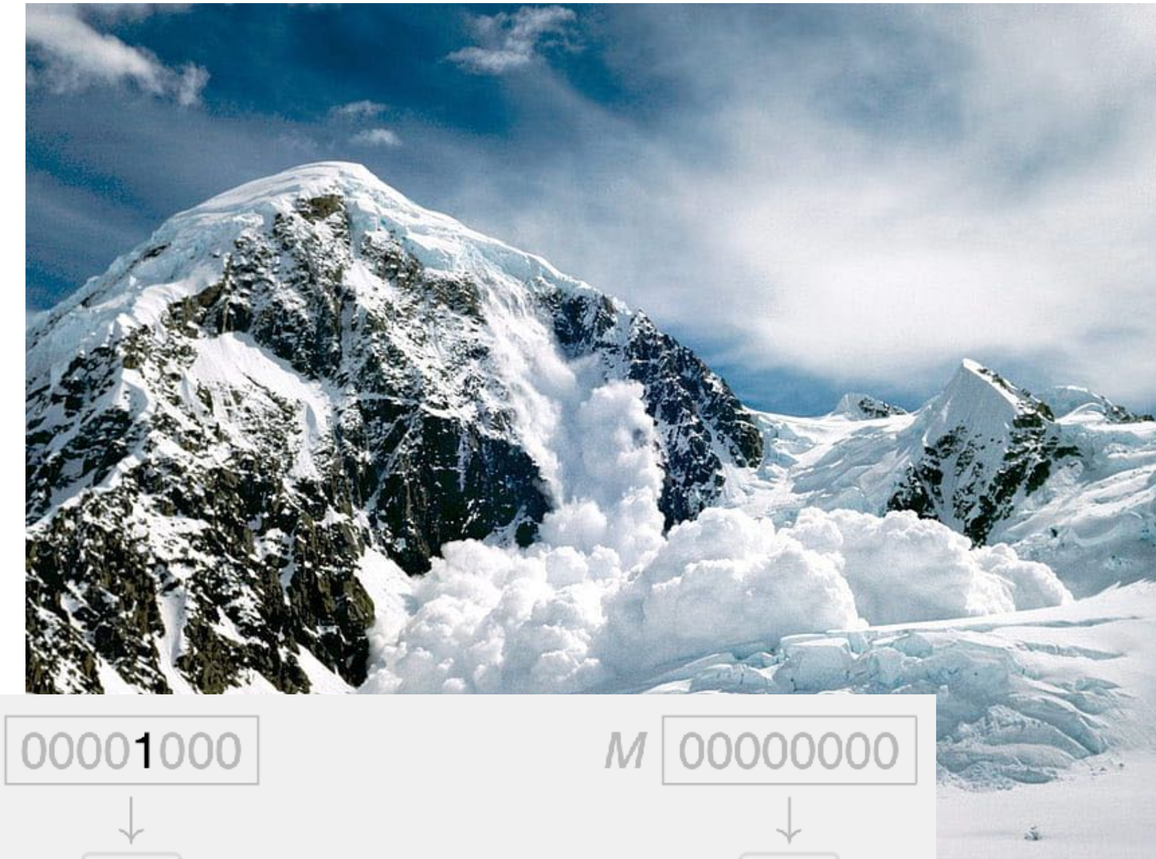
# Why all of This?



If you cannot be a *truly* random permutation, at least strive to look like one.

# What Is the Core Property of Block Ciphers?

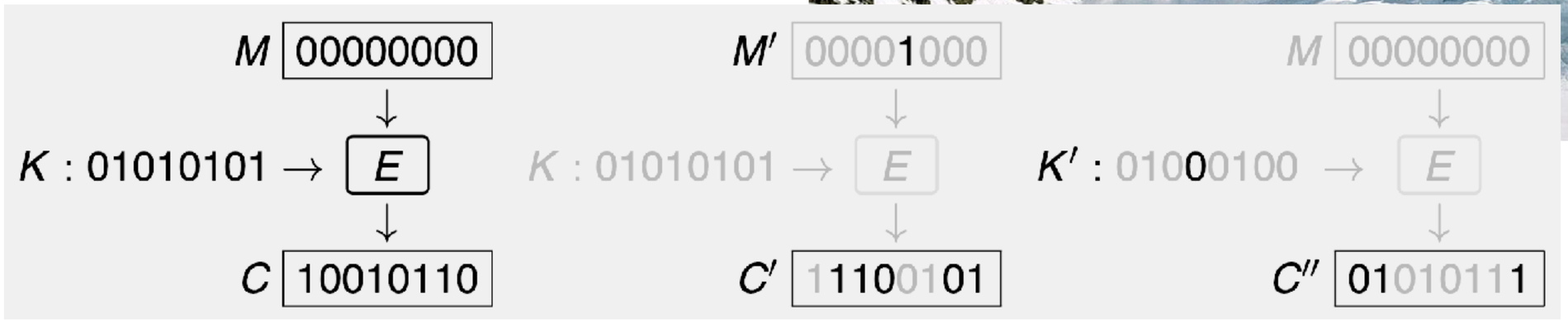
We cannot (efficiently) implement (all possible) random permutations for reasonable sizes of *n*.
Instead, we strive to construct ciphers that cannot be distinguished from random permutations



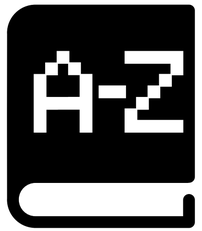
**Desired Property: Avalanche effect**

A small change in either *m* or *k* should result in a large "unpredictable" change in $c = E(k, m)$.

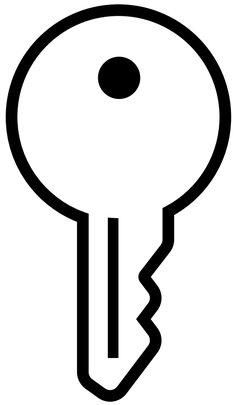
A change of one input bit should change *about half* the bits of output. (If all bits changed, that would be predictable...)



# A Note on Key Sizes




A block cipher with an $n$-bit key is vulnerable to brute-force attack in which $\mathscr{A}$ enumerates all the $2^n$ possible keys until it finds the right one.



So in order to achieve security against attackers running in time $2^n$ we need to use secret keys that are at least $n + 1$ bits long.

# TDA352/DIT352 CRYPTOGRAPHY

## Module 1 : Lecture 2 - Agenda

**Pseudorandom Generators (PRG)**

- Security (Real or Random?)
- Secure encryption from PRG
- Semantic Security (Left or Right)
- PRG-Based Encryption [Proof]

**Block Ciphers**

- Pseudo Random Permutations
- Definition of Block Cipher
- Chosen Plaintext Attack (IND-CPA)
- AES (Advanced Encryption Standard)

**Modes of Operation**

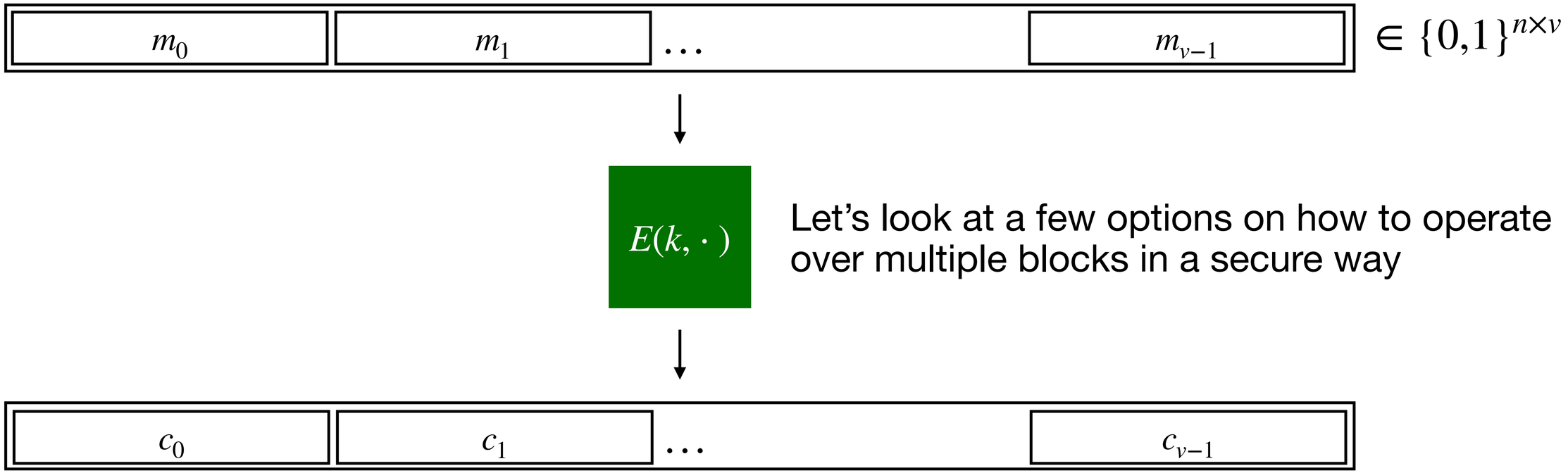
- ECB, CBC, CTR
- Is AES-ECB IND-CPA?
- Message Padding

# Encrypting Long Messages (or Multiple Messages)

## With the Same Key, *Securely*

A Block Cipher is a **deterministic**, **keyed** function that is **invertible**.

$$E(k, \cdot) : \{0,1\}^n \rightarrow \{0,1\}^n$$




Let's look at a few options on how to operate over multiple blocks in a secure way

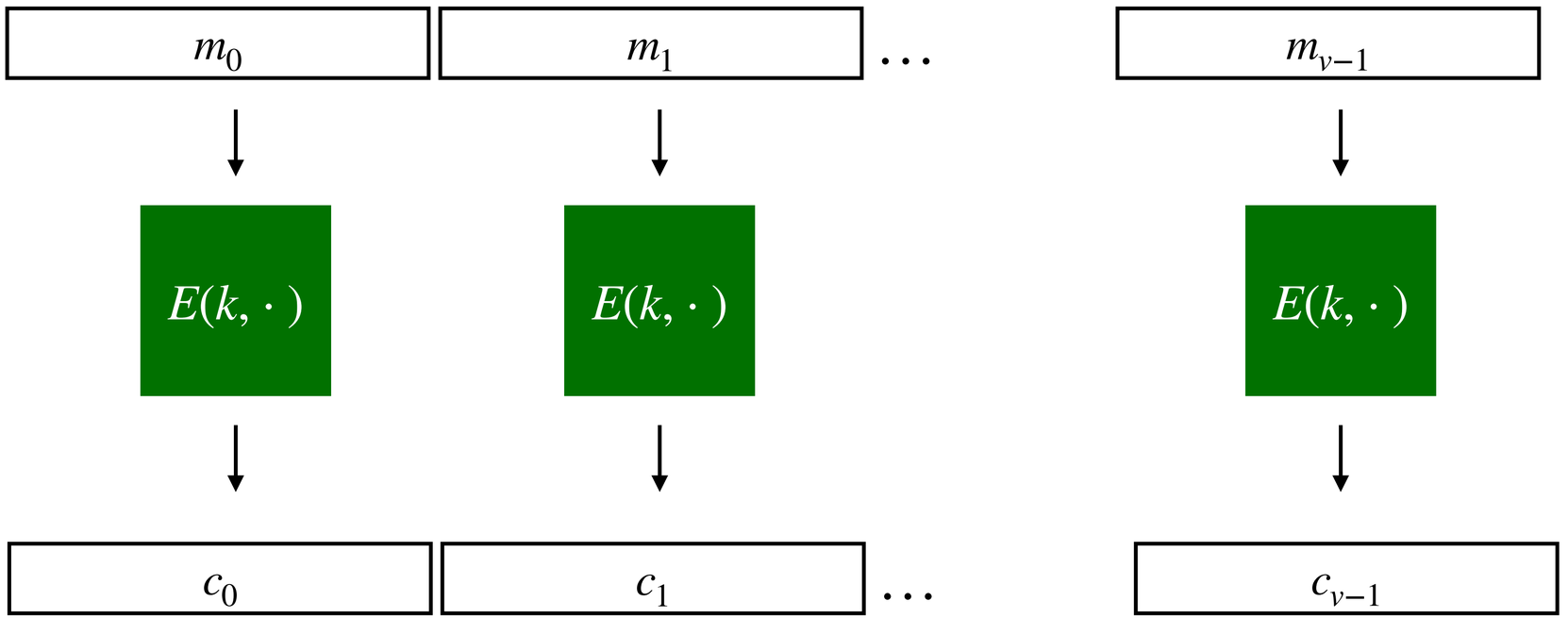
# Electronic Code Book Mode (ECB)

+ very easy to understand and implement
+ encryption and decryption are parallelizable (important for large data)
- lacks diffusion (it encrypts identical plaintext blocks into identical ciphertext blocks)

**ECB is not recommended for use in cryptographic protocols.**

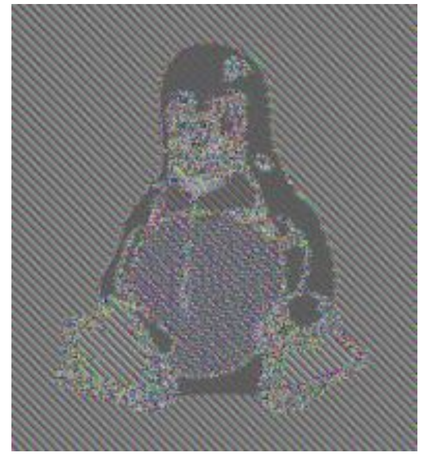


AES – ECB




🧐 *How can we express this with a formula?*

$$c_i = E(k, m_i), \quad m_i = D(k, c_i), \quad for\ i = 0, \ldots, v-1$$



# Cipher Block Chaining Mode (CBC)

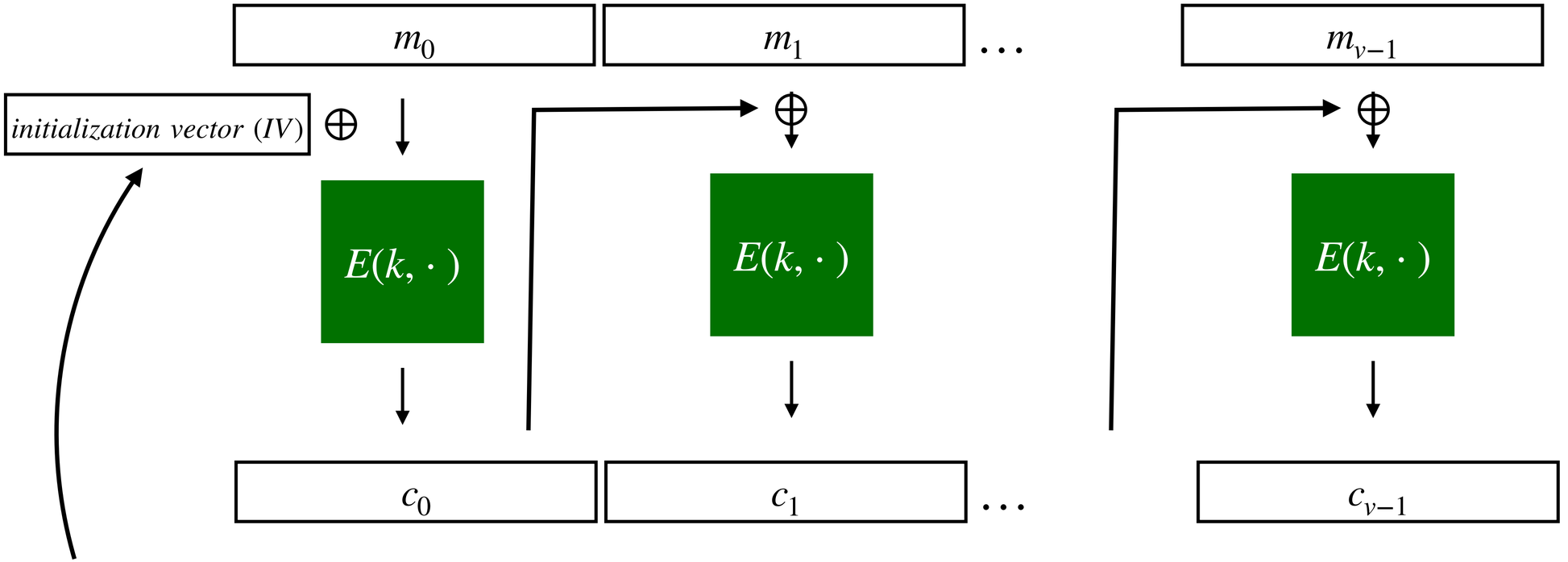
🧐 *formula?*

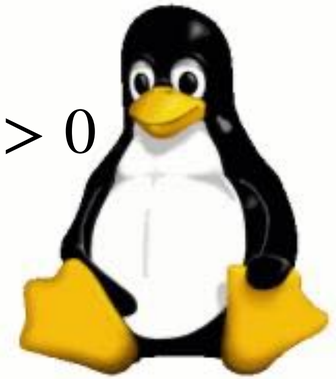
+ Decryption is parallelizable
- Encryption is sequential (not parallelizable)

**CBC is OK to use, but there's better.**

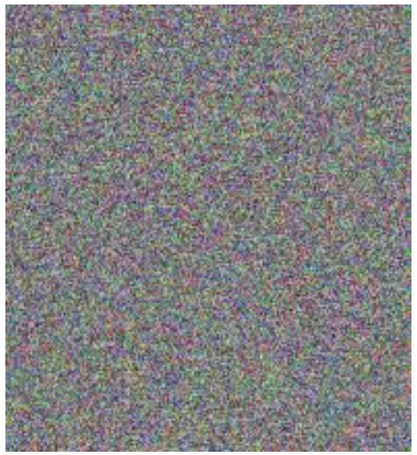
$$c_0 = E(k, m_0 \oplus IV)$$
$$c_i = E(k, m_i \oplus c_{i-1}) \; for \; i > 0$$

$$m_0 = D(k, c_0) \oplus IV$$
$$m_i = D(k, c_i) \oplus c_{i-1} \; for \; i > 0$$



AES – CBC



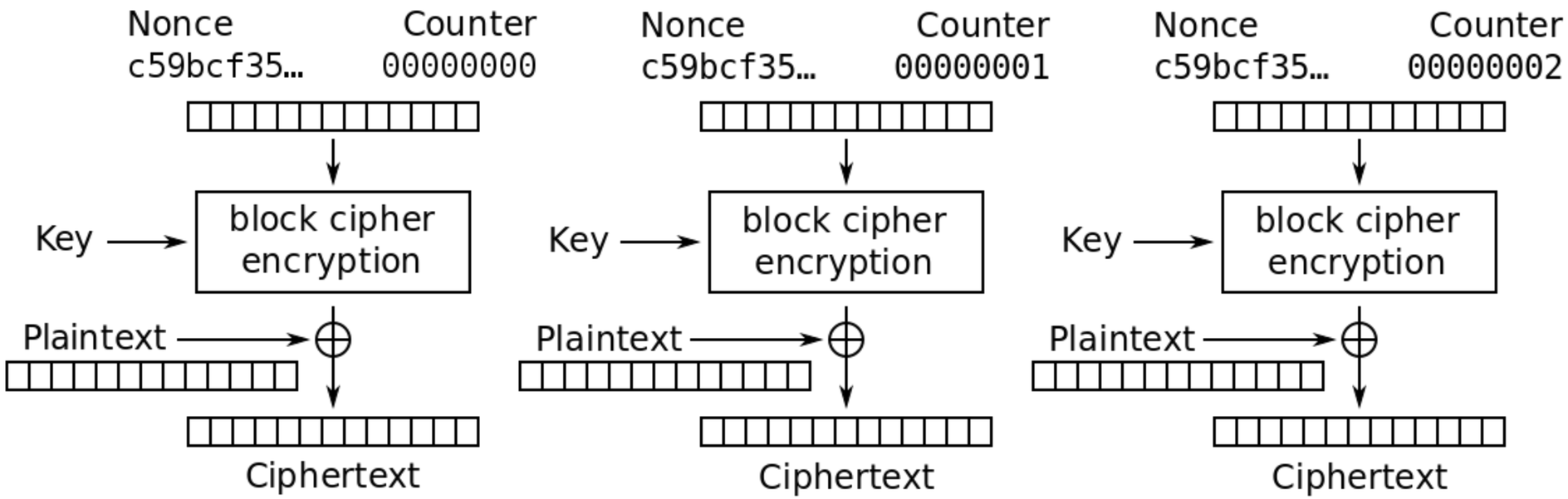
*IV is the source of randomness*

*IV needs to travel with the ciphertext to enable the decryption of* $c_0$

# Counter Mode (CTR)

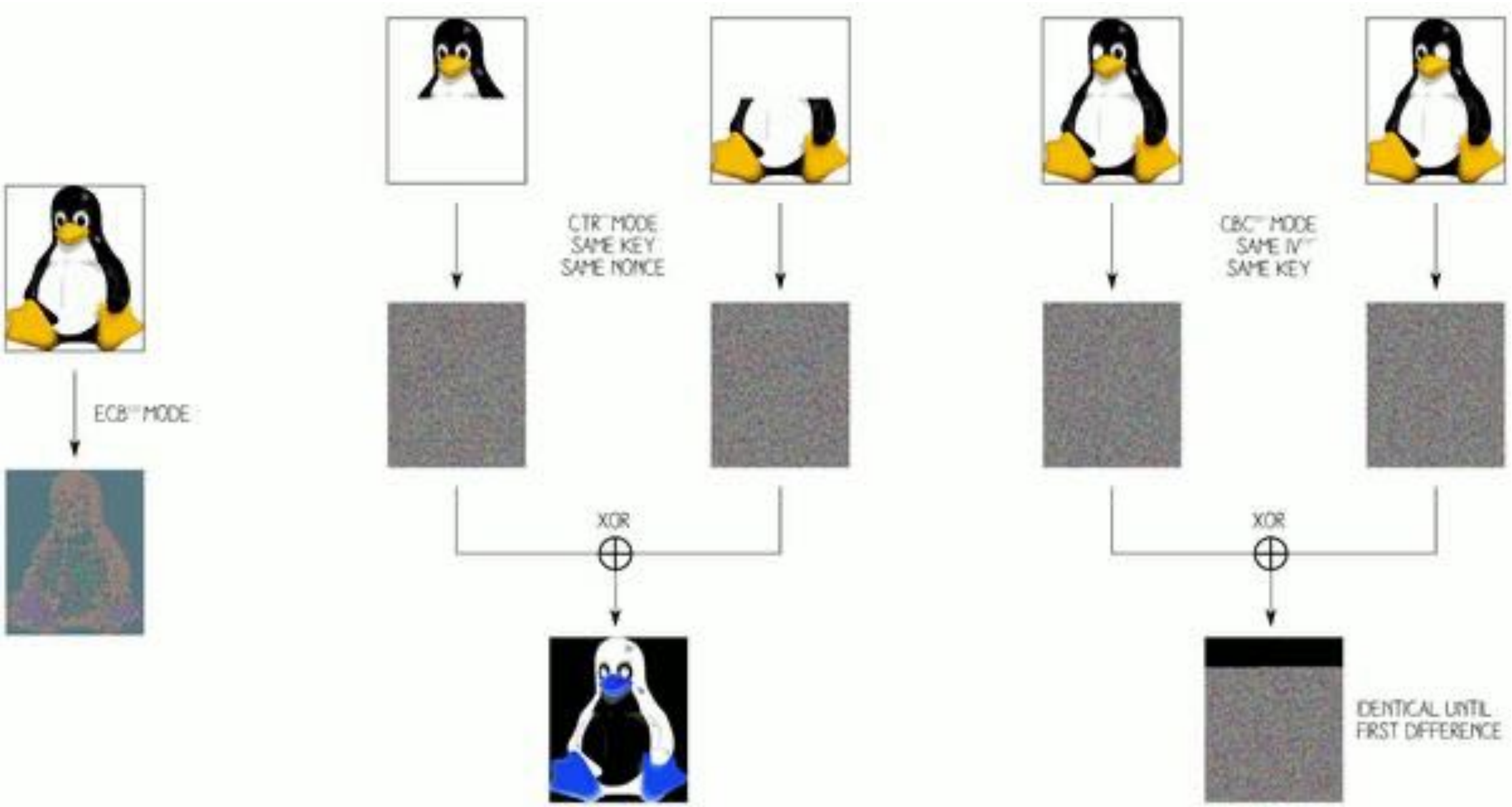
+ Encryption and Decryption are parallelizable (and basically E=D!)
- The nonce value is needed to decrypt every ciphertext block

**CTR is OK to use, with care: nonce = n**umber used **once…otherwise…**



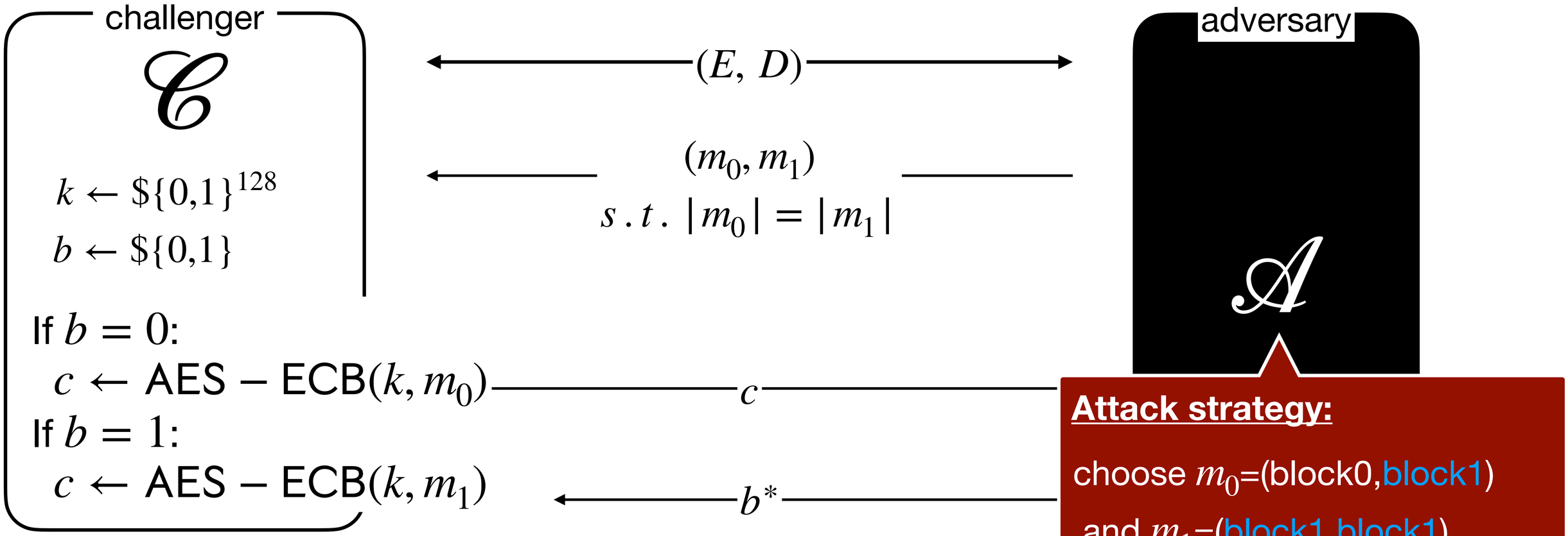

Counter (CTR) mode encryption

# Modes of Operation's Failures - Visual Examples



Now you have the basics to tackle Tasks 1, 2, and 3 in HA1

# Is AES-ECB IND-CPA Secure (Semantically Secure)?




**Attack strategy:**

choose $m_0$=(block0,block1)

and $m_1$=(block1,block1)

Note that $c = (c_0, c_1)$

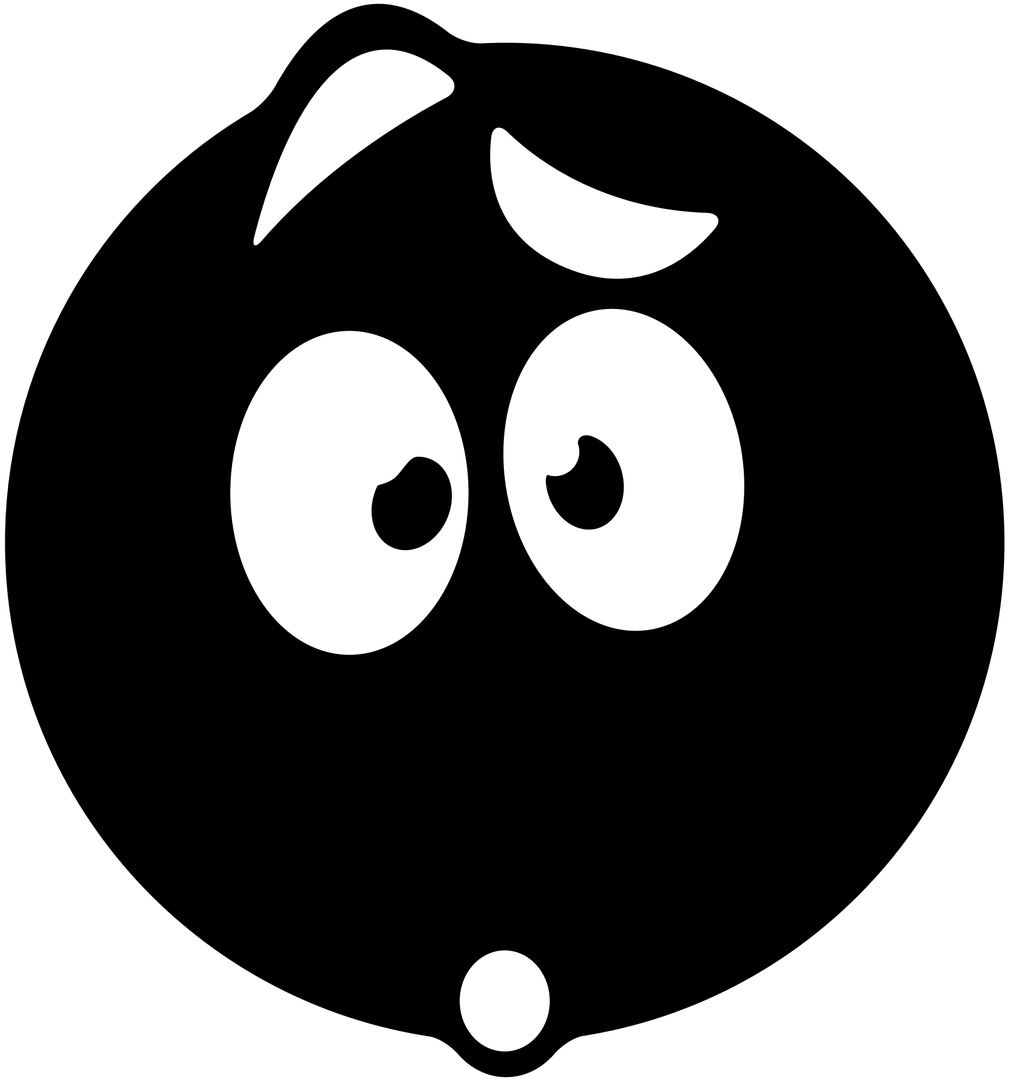
If $c_0 = c_1$ return $b^* = 1$

If $c_0 \neq c_1$ return $b^* = 0$

🧐 *What is the winning condition?*

$\mathscr{A}$ wins the security game iff $b^* = b$

If $b^* \neq b$, $\mathscr{A}$ loses the game



# Are other modes of operation “secure”?

**Yes!** and they also satisfy a new, stronger security notion

# Lessons Learned So Far



From ECB: **deterministic encryption is not semantically secure**.
(if the same key is used to encrypt the same block, the resulting ciphertext will always be the same)

CBC and CTR use **randomness** (IV and nonce respectively), so two encryptions of the same plaintext under the same key will (generally) produce two different ciphertexts

# We Want *Randomized (Aka Probabilistic)* Encryption!

Bad news: probabilistic encryption generates ciphertexts **larger** than the plaintext
(this is an inevitable price to pay for *better* security)

Good news: in certain settings we can get *good* security **without ciphertext expansion**

# Is AES-CBC or AES-CTR Mode IND-CPA Secure?

**Yes**, both are! But we'll skip the formal proofs (too technical for this course)

*I didn't formally define what a secure block cipher is… for the purpose of this course this corresponds to (E,D) being indistinguishable from a random permutation over the block space* $\{0,1\}^n$

If (E,D) is a **secure block cipher**, then:

- using (E,D) in **CBC** mode yields an **IND-CPA** secure cipher
- using (E,D) in **CTR** mode yields an **IND-CPA** secure cipher

*It is not possible to mathematically prove a block cipher to be secure, instead, confidence on its security builds up over years of scrutiny*

# A Word on Padding for Block Ciphers

Padding of messages is often needed before encryption with a block cipher.
(A message must be as long as a integer number of full blocks)

- Essential property: padding must be **reversible**, i.e., the receiver must be able to remove padding in a **unique** way.
- Example of **insecure** padding: append **zeros** to fill last block.
- The receiver should always check that the padding is correctly applied when removing it. In case of mismatch, the protocol should be immediately aborted.

# A Standard Padding : PKCS#7

"N bytes of value N" e.g. 0x04040404

N *ranges from 1 to the block length*